



Trilha AWS

Plano técnico para evolução até nível sênior

Cloud Architecture | DevOps | EKS | Segurança | FinOps

Objetivo da trilha

Formar domínio cloud para projetar, operar, proteger, automatizar e otimizar workloads reais na AWS, saindo do nível Associate para uma prática Profissional/Sênior com foco em produção.

Nível: Avançado/Sênior

Arquitetura

Operação

Segurança

FinOps

Sumário

Navegação estrutural do documento com os principais módulos, áreas de domínio e projetos.

1. Análise inicial da trilha	3
2. Classificação por nível	4
3. Estrutura progressiva da trilha	6
Módulo 1 — Fundamentos AWS	6
Módulo 2 — Arquitetura AWS Associate forte	9
Módulo 3 — Desenvolvimento AWS	16
Módulo 4 — Operação AWS / CloudOps	20
Módulo 5 — DevOps Professional, EKS, segurança avançada e FinOps	22
5.1 DevOps Professional — DOP-C02	22
5.2 EKS e cloud-native AWS	25
5.3 Segurança AWS avançada	27
5.4 FinOps AWS	29
Módulo 6 — Domínio sênior e liderança técnica	31
4. Critérios de domínio técnico por grande área	33
5. Aplicações práticas em sistemas reais	35
6. Plano de execução sugerido	38
7. Projetos recomendados para portfólio	43
8. Checklist de domínio técnico	45
9. Indicadores de prontidão para nível avançado/sênior	47
10. Resumo final da trilha	47

Trilha AWS - Plano técnico para evolução até nível sênior

Plano de estudos estruturado para domínio cloud, arquitetura, desenvolvimento, operação, DevOps, EKS, segurança e FinOps.

Foco Arquitetura, operação, automação, segurança e otimização AWS.	Nível-alvo Associate forte até prática Profissional/Sênior.
Projetos API Java, Serverless, CI/CD cross-account, EKS, Segurança e FinOps.	Resultado Capacidade de projetar, justificar, operar e evoluir workloads críticos.

1. Análise inicial da trilha

Tema central

A trilha é uma formação completa em cloud computing com AWS, cobrindo fundamentos, arquitetura, desenvolvimento cloud-native, operação, DevOps, Kubernetes/EKS, segurança avançada e FinOps.

O foco não é apenas “conhecer serviços AWS”, mas desenvolver capacidade de projetar, implementar, operar, proteger, automatizar e otimizar workloads reais em produção.

Objetivo principal

Formar um profissional capaz de sair do nível Associate e evoluir para prática Profissional/Sênior, com domínio suficiente para:

- desenhar arquiteturas resilientes;
- justificar decisões técnicas;
- reduzir custos sem comprometer confiabilidade;
- automatizar ambientes e deploys;
- estruturar contas AWS com governança;
- operar incidentes em produção;
- aplicar segurança por padrão;
- liderar decisões cloud em sistemas reais.

A AWS organiza parte dessa evolução pelas certificações. A Cloud Practitioner valida entendimento fundamental da nuvem e dos serviços AWS; a Solutions Architect Associate foca desenho de soluções otimizadas por custo e performance; a Developer Associate valida desenvolvimento, deploy, otimização e troubleshooting de aplicações AWS; e a DevOps Engineer Professional valida proficiência avançada em provisionamento, operação e gestão de sistemas distribuídos na AWS.

Observação importante sobre SOA-C03

No conteúdo enviado aparece SOA-C03 associado à operação AWS. A AWS atualizou essa trilha: o antigo AWS Certified SysOps Administrator - Associate foi atualizado para AWS Certified CloudOps Engineer - Associate, com o exame SOA-C03 disponível a partir de 30 de setembro de 2025. Portanto, nesta trilha, o módulo de operação deve ser tratado como CloudOps Engineer Associate — SOA-C03, não mais como SysOps tradicional.

2. Classificação por nível

Básico

Aqui o estudante precisa entender como a nuvem funciona.

Conteúdos principais:

- Regions;
- Availability Zones;
- Edge Locations;
- modelo de responsabilidade compartilhada;
- IAM básico;
- billing;
- Organizations básico;
- CloudTrail;
- CloudWatch;
- Well-Architected Framework;
- pilares de segurança, confiabilidade, performance, custo, operação e sustentabilidade.

Nível esperado: conseguir explicar o funcionamento da AWS, entender cobrança, segurança básica, disponibilidade e os principais conceitos de arquitetura.

Intermediário

Aqui o estudante começa a desenhar e implementar arquiteturas reais.

Conteúdos principais:

- VPC;
- subnets públicas e privadas;
- roteamento;
- NAT Gateway;
- Internet Gateway;
- Security Groups;
- NACLs;
- EC2;
- Load Balancer;
- Auto Scaling;

- Route 53;
- S3;
- CloudFront;
- RDS;
- Aurora;
- DynamoDB;
- SQS;
- SNS;
- EventBridge;
- Lambda;
- API Gateway;
- KMS;
- Secrets Manager;
- ACM;
- WAF;
- Backup;
- DR.

Nível esperado: conseguir desenhar uma aplicação web/API altamente disponível, segura e com custo controlado.

Avançado

Aqui o estudante aprende a desenvolver, automatizar e operar workloads cloud-native.

Conteúdos principais:

- serverless;
- Step Functions;
- SAM;
- CDK;
- pipelines;
- CodeBuild;
- CodeDeploy;
- CodePipeline;
- X-Ray;
- CloudWatch Logs;
- IAM least privilege;
- Parameter Store;
- troubleshooting;
- Config;
- Systems Manager;
- incident response.

Nível esperado: conseguir construir aplicações orientadas a eventos, automatizar infraestrutura e operar falhas reais em produção.

Sênior

Aqui o profissional deixa de apenas executar e passa a decidir, projetar, revisar e liderar.

Conteúdos principais:

- DevOps Profissional;
- multi-account;
- cross-account deployment;
- blue/green;
- canary;
- rolling deployment;
- CloudFormation avançado;
- StackSets;
- drift detection;
- CDK Pipelines;
- EKS;
- Karpenter;
- IRSA;
- GitOps com ArgoCD;
- observabilidade;
- segurança multi-account;
- SCPs;
- Control Tower;
- GuardDuty;
- Security Hub;
- Inspector;
- Macie;
- CloudTrail Lake;
- FinOps;
- unit economics.

Nível esperado: saber justificar arquitetura, custo, risco, segurança, operação e evolução técnica de workloads críticos.

3. Estrutura progressiva da trilha

Módulo 1 — Fundamentos AWS

Objetivo do módulo

Construir base sólida em cloud computing, serviços essenciais, modelo de segurança, cobrança, governança inicial e princípios de arquitetura.

A AWS Well-Architected Framework usa seis pilares: Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization e Sustainability. Esses pilares devem ser tratados como base de decisão durante toda a trilha, não como teoria isolada.

Conteúdos a estudar

- Regions;
- Availability Zones;
- Edge Locations;
- Shared Responsibility Model;
- IAM;
- Billing;
- AWS Organizations;
- CloudTrail;
- CloudWatch;
- Well-Architected Framework;
- Reliability;
- Security;
- Performance Efficiency;
- Cost Optimization;
- Operational Excellence;
- Sustainability.

Explicação técnica dos tópicos

Regions são localizações geográficas onde a AWS mantém infraestrutura física. Servem para aproximar workloads dos usuários, cumprir requisitos regulatórios e reduzir latência. Em projetos reais, a escolha da região impacta custo, latência, disponibilidade de serviços e compliance.

Erro comum: escolher região apenas por preço, ignorando latência, LGPD, disponibilidade de serviços e estratégia de disaster recovery.

Visão sênior: decidir região exige avaliar usuários, dados sensíveis, custo de egress, requisitos legais, multi-region, RTO e RPO.

Availability Zones são zonas isoladas dentro de uma região. Servem para alta disponibilidade. Em produção, bancos, balanceadores, aplicações e caches devem ser distribuídos em múltiplas AZs quando o workload exige resiliência.

Erro comum: subir tudo em uma única AZ e acreditar que “estar na AWS” já garante alta disponibilidade.

Visão sênior: saber quando Multi-AZ é obrigatório, quando é desperdício de custo e quando uma estratégia multi-region é realmente necessária.

Edge Locations são pontos de presença usados por serviços como CloudFront. Servem para cache, distribuição de conteúdo e redução de latência.

Erro comum: usar CloudFront apenas para arquivos estáticos e ignorar cache de APIs, headers, compressão, TLS e proteção de borda.

Visão sênior: projetar cache invalidation, políticas de TTL, origem privada, WAF na borda e estratégia de redução de egress.

Shared Responsibility Model define o que é responsabilidade da AWS e o que é responsabilidade do cliente. A AWS protege a infraestrutura da nuvem; o cliente protege o que configura e executa na nuvem.

Erro comum: achar que a AWS é responsável por permissões IAM mal configuradas, buckets públicos, secrets vazados ou aplicações vulneráveis.

Visão sênior: transformar esse modelo em checklist de arquitetura, segurança, operação, compliance e revisão de produção.

IAM controla identidade e acesso. Serve para definir quem pode fazer o quê, em qual recurso e sob quais condições.

Erro comum: usar AdministratorAccess, usuário IAM fixo, access key longa, permissões curingas e roles sem boundary.

Visão sênior: aplicar least privilege, roles temporárias, permission boundaries, ABAC, separação por ambiente, revisão contínua e auditoria de permissões.

Billing permite entender custos. Serve para evitar surpresas financeiras, prever gastos e associar consumo a ambientes, times, features e produtos.

Erro comum: estudar AWS sem configurar budgets, tags e alarmes de custo.

Visão sênior: custo é requisito arquitetural. Toda decisão técnica precisa considerar custo fixo, custo variável, tráfego, armazenamento, licenças, ociosidade e crescimento.

AWS Organizations organiza múltiplas contas. Serve para separar ambientes, centralizar governança e aplicar políticas. Em ambientes sênior, contas separadas para dev, staging, prod, security, audit e shared services são padrão.

Erro comum: usar uma única conta para tudo.

Visão sênior: desenhar landing zone multi-account com OUs, SCPs, logs centralizados, billing consolidado e segregação de responsabilidades.

CloudTrail registra eventos de API. Serve para auditoria, investigação e rastreabilidade.

Erro comum: ativar apenas no básico e não centralizar logs em conta de auditoria.

Visão sênior: CloudTrail deve estar integrado a detecção, resposta a incidentes, trilhas organizacionais, retenção e análise forense.

CloudWatch coleta métricas, logs e alarmes. Serve para observabilidade operacional.

Erro comum: só olhar CPU e memória, sem métricas de negócio, latência, erro, saturação e filas.

Visão sênior: definir SLIs, SLOs, alarmes acionáveis e dashboards por criticidade.

Ordem recomendada

- Cloud concepts.
- Regions, AZs e Edge.
- IAM e responsabilidade compartilhada.
- Billing, budgets e tags.
- CloudWatch e CloudTrail.

- Well-Architected Framework.

Exercícios técnicos

- Criar uma conta AWS com MFA.
- Criar IAM users/roles com permissões mínimas.
- Configurar Budget mensal.
- Ativar CloudTrail.
- Criar alarmes simples no CloudWatch.
- Fazer uma revisão Well-Architected de uma aplicação fictícia.

Mini projeto

- Criar uma landing zone básica:
- conta organizada;
- MFA;
- usuário sem root;
- budget;
- CloudTrail;
- CloudWatch alarm;
- bucket S3 privado para logs;
- tags obrigatórias.

Critérios de conclusão

- Você conclui este módulo quando conseguir explicar:
- diferença entre Region e AZ;
- como IAM avalia permissões;
- o que é responsabilidade da AWS e do cliente;
- como auditar ações;
- como detectar aumento de custo;
- como aplicar os seis pilares do Well-Architected.
- Nível esperado
- Básico forte, pronto para iniciar arquitetura Associate.

Módulo 2 — Arquitetura AWS Associate forte

Objetivo do módulo

Aprender a projetar aplicações reais com rede, compute, storage, banco de dados, mensageria, segurança, CDN, balanceamento, backup e disaster recovery.

Este módulo se conecta diretamente à AWS Certified Solutions Architect - Associate, que foca o desenho de soluções otimizadas por custo e performance.

Conteúdos a estudar

- VPC;

- subnets públicas e privadas;
- Route Tables;
- NAT Gateway;
- Internet Gateway;
- Security Groups;
- NACLs;
- EC2;
- AMI;
- Auto Scaling;
- Load Balancer;
- ALB/NLB;
- Route 53;
- S3;
- CloudFront;
- RDS;
- Aurora;
- DynamoDB;
- ElastiCache;
- SQS;
- SNS;
- EventBridge;
- Lambda;
- API Gateway;
- EFS;
- EBS;
- KMS;
- Secrets Manager;
- ACM;
- WAF;
- Shield;
- Backup;
- DR.

Explicação técnica dos principais blocos

Rede: VPC, subnets, rotas, NAT, IGW, SG e NACL

VPC é a rede isolada do workload. É onde você define blocos CIDR, subnets, roteamento, gateways e conectividade.

Serve para controlar isolamento, exposição pública, comunicação interna e integração com outros ambientes.

Em projetos reais, a VPC separa camadas como frontend, backend, banco de dados, cache e componentes privados.

Problemas que resolve:

- isolamento de rede;
- controle de tráfego;
- segmentação por camada;
- conectividade privada;
- redução de superfície de ataque.

Erros comuns:

- CIDR mal planejado;
- subnet pública para banco de dados;
- liberar 0.0.0.0/0 sem necessidade;
- confundir Security Group com NACL;
- não planejar crescimento da rede;
- criar NAT Gateway sem analisar custo.

Conhecimentos prévios:

- IP;
- CIDR;
- TCP/UDP;
- portas;
- DNS;
- roteamento;
- firewall básico.

Nível sênior esperado:

- desenhar rede por ambiente;
- projetar conectividade híbrida;
- justificar NAT Gateway vs VPC Endpoint;
- controlar tráfego leste-oeste;
- definir segmentação segura;
- evitar sobreposição de CIDR;
- revisar custos de tráfego e NAT.
- Compute: EC2, AMI, Auto Scaling, ALB e NLB

EC2 é computação baseada em máquinas virtuais. AMI é a imagem base da instância. Auto Scaling ajusta capacidade. ALB/NLB distribuem tráfego.

Servem para hospedar aplicações, APIs, workers, gateways, aplicações legadas e serviços que exigem controle de sistema operacional.

Em projetos reais, EC2 ainda é comum em aplicações Java Spring Boot, workers batch, sistemas legados, aplicações containerizadas sem Kubernetes e integrações internas.

Problemas que resolve:

- execução de workloads customizados;
- escalabilidade horizontal;
- alta disponibilidade;

- balanceamento de tráfego;
- controle fino de runtime.

Erros comuns:

- instância superdimensionada;
- AMI sem patch;
- não usar Auto Scaling;
- health check mal configurado;
- expor instância diretamente sem Load Balancer;
- armazenar estado local sem estratégia.

Nível sênior esperado:

- escolher família de instância por workload;
- usar launch templates;
- definir scaling policies;
- aplicar blue/green ou rolling;
- integrar logs e métricas;
- projetar rollback;
- calcular custo por capacidade.
- DNS e entrega: Route 53, CloudFront, ACM, WAF e Shield

Route 53 gerencia DNS. CloudFront distribui conteúdo via CDN. ACM gerencia certificados TLS. WAF filtra tráfego HTTP. Shield protege contra DDoS.

Servem para expor aplicações com baixa latência, TLS, proteção de borda e roteamento resiliente.

Em projetos reais, aparecem em APIs públicas, frontends Next.js/React, e-commerces, portais, aplicações SaaS e landing pages corporativas.

Problemas que resolve:

- latência global;
- proteção de borda;
- TLS gerenciado;
- failover DNS;
- caching;
- mitigação de ataques comuns.

Erros comuns:

- não configurar HTTPS corretamente;
- cachear conteúdo sensível;
- invalidar cache de forma descontrolada;
- não usar WAF em endpoints críticos;
- não proteger origem.

Nível sênior esperado:

- projetar CDN com origem privada;
- aplicar WAF por risco real;
- criar políticas de cache;

- usar failover ponderado/geográfico;
- analisar impacto de TTL;
- reduzir custo de egress.
- Storage: S3, EBS e EFS
- S3 é armazenamento de objetos. EBS é armazenamento em bloco para EC2. EFS é sistema de arquivos compartilhado.
- Servem para armazenar arquivos, backups, mídia, logs, artefatos, volumes de instâncias e dados compartilhados.
- Em projetos reais:
 - S3 para uploads, relatórios, imagens e data lake;
 - EBS para disco de instância;
 - EFS para compartilhamento entre instâncias ou containers.

Problemas que resolve:

- armazenamento durável;
- separação de arquivos da aplicação;
- backup;
- lifecycle;
- compartilhamento;
- escalabilidade de objetos.

Erros comuns:

- bucket público sem necessidade;
- não configurar lifecycle;
- usar EBS para arquivos que deveriam estar no S3;
- não criptografar dados;
- não versionar objetos críticos.

Nível sênior esperado:

- escolher storage por padrão de acesso;
- aplicar versionamento, lifecycle e criptografia;
- desenhar retenção;
- controlar acesso com bucket policy;
- calcular custo por classe de armazenamento;
- projetar recuperação contra exclusão acidental.
- Bancos: RDS, Aurora, DynamoDB e ElastiCache

RDS gerencia bancos relacionais. Aurora é banco relacional otimizado para cloud. DynamoDB é NoSQL chave-valor/documento de alta escala. ElastiCache fornece cache Redis/Memcached.

Servem para persistência transacional, leitura escalável, cache, baixa latência e alta disponibilidade.

Em projetos reais:

- RDS/Aurora para sistemas financeiros, pedidos, usuários e transações;
- DynamoDB para eventos, sessões, catálogos, idempotência e serverless;
- Redis para cache, rate limiting, sessão e locks distribuídos.

Problemas que resolve:

- persistência confiável;
- escalabilidade de leitura;
- baixa latência;
- desacoplamento de carga;
- redução de pressão no banco principal.

Erros comuns:

- escolher DynamoDB sem modelar access patterns;
- usar cache sem estratégia de invalidação;
- não definir backup;
- subdimensionar conexões;
- ignorar índices;
- não planejar migração.

Nível sênior esperado:

- decidir relacional vs NoSQL;
- modelar particionamento;
- definir estratégia de backup/restore;
- controlar conexão em aplicações Java;
- projetar read replicas;
- avaliar consistência;
- justificar custo de Aurora vs RDS;
- aplicar cache sem quebrar consistência.
- Mensageria e eventos: SQS, SNS e EventBridge
- SQS é fila. SNS é pub/sub. EventBridge é barramento de eventos.
- Servem para desacoplar serviços, absorver picos, processar assíncrono e criar arquiteturas orientadas a eventos.
- Em projetos reais:
- fila de pedidos;
- processamento de pagamento;
- envio de e-mail;
- integração entre microsserviços;
- eventos de domínio;
- workflows assíncronos.

Problemas que resolve:

- acoplamento forte;
- indisponibilidade transitória;
- picos de carga;
- retry;
- fan-out;
- comunicação assíncrona.

Erros comuns:

- não configurar DLQ;

- não tratar idempotência;
- usar fila como banco;
- ignorar ordering;
- não monitorar backlog;
- payload grande demais.

Nível sênior esperado:

- decidir SQS vs SNS vs EventBridge;
- definir contrato de eventos;
- projetar retries e DLQ;
- aplicar idempotência;
- calcular throughput;
- evitar cascata de falhas;
- rastrear eventos ponta a ponta.
- Segurança e criptografia: KMS, Secrets Manager, ACM, WAF e Shield

KMS gerencia chaves criptográficas. Secrets Manager armazena segredos com rotação. ACM gerencia certificados. WAF e Shield protegem aplicações públicas.

Servem para proteger dados, credenciais, tráfego e endpoints.

Erros comuns:

- secrets no código;
- .env com segredo em repositório;
- chaves compartilhadas sem rotação;
- permissões amplas para descriptografar;
- ignorar proteção de APIs públicas.

Nível sênior esperado:

- aplicar criptografia em repouso e trânsito;
- desenhar rotação de segredo;
- usar IAM conditions;
- restringir decrypt por contexto;
- auditar acesso a segredos;
- proteger borda com WAF.
- Backup e DR
- Backup protege contra perda de dados. DR define recuperação após falhas graves.

Conceitos obrigatórios:

- RTO;
- RPO;
- backup;
- restore;
- snapshot;
- multi-AZ;
- multi-region;

- pilot light;
- warm standby;
- active-active.

Erro comum: ter backup, mas nunca testar restore.

Nível sênior esperado:

- definir RTO/RPO por criticidade;
- testar recuperação;
- automatizar backup;
- simular falhas;
- documentar runbooks;
- justificar custo da estratégia.

Mini projeto

- Arquitetura de API Java Spring Boot em AWS;
- VPC com subnets públicas e privadas;
- ALB público;
- EC2/ECS privado;
- RDS Multi-AZ;
- S3 para arquivos;
- CloudFront para distribuição;
- Route 53 + ACM;
- Secrets Manager;
- CloudWatch;
- Backup;
- WAF;
- SQS para processamento assíncrono.

Critério sênior

Saber desenhar uma arquitetura altamente disponível, segura, observável e financeiramente justificável.

Módulo 3 — Desenvolvimento AWS

Objetivo do módulo

Construir aplicações cloud-native usando serverless, eventos, CI/CD, observabilidade e infraestrutura como código.

Este módulo se conecta à AWS Certified Developer - Associate, que valida desenvolvimento, otimização, empacotamento, deploy, CI/CD e resolução de problemas em aplicações AWS.

Conteúdos a estudar

- Lambda;

- API Gateway;
- DynamoDB;
- S3 Events;
- SQS;
- SNS;
- EventBridge;
- Step Functions;
- SAM;
- CDK;
- CodeBuild;
- CodeDeploy;
- CodePipeline;
- X-Ray;
- CloudWatch Logs;
- Powertools;
- IAM least privilege;
- Secrets Manager;
- Parameter Store.

Explicação técnica

Lambda executa código sob demanda sem gerenciar servidor. Serve para APIs, workers, processamento de eventos, automações e integrações.

Erros comuns:

- função grande demais;
- timeout mal definido;
- cold start ignorado;
- ausência de idempotência;
- acoplamento excessivo;
- permissões amplas.

Visão sênior:

- decidir Lambda vs ECS vs EKS;
- otimizar cold start;
- controlar concorrência;
- projetar retries;
- isolar responsabilidades;
- monitorar custo por invocação.
- API Gateway expõe APIs HTTP/REST/WebSocket. Serve para autenticação, throttling, integração com Lambda e controle de tráfego.

Erros comuns:

- não configurar rate limit;
- expor endpoints sem autenticação;

- ignorar payload size;
- não versionar APIs.

Visão sênior:

- projetar contrato de API;
- aplicar throttling por cliente;
- proteger com authorizer;
- usar logs estruturados;
- definir estratégia REST vs HTTP API.
- DynamoDB exige modelagem por access pattern. Não se modela DynamoDB como banco relacional.

Erros comuns:

- criar tabela antes de entender consultas;
- usar scan em produção;
- chave de partição com baixa cardinalidade;
- ignorar hot partitions.

Visão sênior:

- desenhar PK/SK;
- GSIs/LSIs;
- single-table design quando fizer sentido;
- TTL;
- streams;
- capacidade on-demand vs provisioned;
- custo por leitura/escrita.
- S3 Events, SQS, SNS e EventBridge habilitam arquitetura orientada a eventos.

Exemplo real:

- usuário faz upload no S3;
- evento dispara Lambda;
- Lambda valida arquivo;
- mensagem vai para SQS;
- worker processa;
- evento de sucesso vai para EventBridge;
- outro serviço atualiza status.

Visão sênior:

- desenhar rastreabilidade;
- evitar duplicidade;
- controlar retries;
- garantir idempotência;
- tratar DLQ;
- versionar eventos.
- Step Functions orquestra fluxos. Serve para coordenar etapas, retries, compensações e integrações.

Erros comuns:

- colocar regra de negócio excessiva na orquestração;
- ignorar custo por transição;
- não tratar falha parcial.

Visão sênior:

- decidir orquestração vs coreografia;
- aplicar saga;
- compensação;
- timeout;
- estado;
- auditoria do fluxo.

SAM e CDK servem para infraestrutura como código. SAM é mais focado em serverless; CDK permite definir infraestrutura usando linguagens de programação.

Visão sênior:

- padronizar constructs;
- separar stacks;
- versionar infraestrutura;
- revisar permissões geradas;
- impedir drift;
- criar pipelines de infraestrutura.
- X-Ray e CloudWatch Logs permitem tracing e análise operacional.

Erro comum: logar sem correlação, sem trace id e sem estrutura.

Visão sênior:

- logs JSON;
- correlation ID;
- tracing distribuído;
- métricas de negócio;
- alarmes por sintoma, não por ruído.

Projeto prático

API serverless completa:

- API Gateway;
- Lambda;
- DynamoDB;
- SQS;
- EventBridge;
- Step Functions;
- Secrets Manager;
- Parameter Store;
- CloudWatch Logs;

- X-Ray;
- CDK ou SAM;
- CI/CD básico.

Critérios de conclusão

- Você conclui este módulo quando conseguir:
- criar APIs serverless seguras;
- modelar DynamoDB por consulta;
- usar filas e eventos com DLQ;
- aplicar IAM least privilege;
- empacotar e versionar infraestrutura;
- investigar falhas por logs e traces.

Módulo 4 — Operação AWS / CloudOps

Objetivo do módulo

Aprender a operar workloads em produção: monitorar, diagnosticar, corrigir, automatizar, auditar, aplicar patches, responder incidentes e manter disponibilidade.

Conteúdos a estudar

- CloudWatch Metrics;
- CloudWatch Alarms;
- Logs Insights;
- X-Ray;
- Systems Manager;
- Session Manager;
- Patch Manager;
- Automation;
- Parameter Store;
- AWS Config;
- Config Rules;
- CloudTrail;
- Backup;
- Auto Scaling troubleshooting;
- RDS troubleshooting;
- VPC Flow Logs;
- Incident response.

Explicação técnica

CloudWatch Metrics e Alarms monitoram sinais operacionais. Servem para detectar latência alta, erro, saturação, uso de CPU, filas acumuladas, falhas de banco e indisponibilidade.

Erro comum: criar alarmes demais e gerar fadiga operacional.

Visão sênior:

- alarmar sintomas reais;
- diferenciar métrica técnica de métrica de negócio;
- definir severidade;
- integrar alerta com runbook;
- medir MTTR.
- Logs Insights consulta logs. Serve para investigar falhas, padrões de erro, requisições lentas e comportamento anormal.

Erro comum: logs sem estrutura e sem campos pesquisáveis.

Visão sênior:

- padronizar logs com traceId, requestId, customerId, orderId, service, environment, errorCode.

Systems Manager, Session Manager, Patch Manager e Automation permitem gerenciar instâncias sem SSH aberto, aplicar patches e automatizar tarefas.

Erro comum:

- abrir porta SSH para internet;
- manter servidor sem patch;
- executar correção manual sem rastreabilidade.

Visão sênior:

- acesso sem chave SSH;
- automações auditáveis;
- patching por janela;
- runbooks versionados;
- inventário de instâncias.
- AWS Config e Config Rules verificam conformidade da infraestrutura.

Exemplo:

- bucket S3 não pode ser público;
- EBS deve ser criptografado;
- Security Group não pode expor porta sensível;
- CloudTrail deve estar ativo.

Visão sênior:

- usar compliance como código;
- integrar remediação automática;
- criar exceções controladas;
- auditar ambiente multi-account.
- VPC Flow Logs registram tráfego de rede. Servem para diagnóstico de conectividade, segurança e troubleshooting.

Erro comum: tentar resolver problema de rede só olhando aplicação.

Visão sênior:

- correlacionar Flow Logs, SG, NACL, route table, DNS e aplicação.
- Incident response define resposta a falhas e incidentes de segurança.

Visão sênior:

- classificar severidade;
- isolar impacto;
- preservar evidências;
- acionar runbook;
- comunicar stakeholders;
- conduzir postmortem sem culpabilização;
- criar ações preventivas.

Mini projeto

- Montar operação de uma API em produção;
- dashboard CloudWatch;
- alarmes;
- logs estruturados;
- X-Ray;
- VPC Flow Logs;
- Config Rules;
- CloudTrail;
- runbook de incidente;
- backup testado;
- troubleshooting simulado de ALB, EC2, RDS e SQS.

Critério sênior

Saber reduzir MTTR, criar operação previsível e transformar incidentes em melhoria sistêmica.

Módulo 5 — DevOps Professional, EKS, segurança avançada e FinOps

Objetivo do módulo

Dominar automação de entrega, infraestrutura como código, Kubernetes na AWS, segurança multi-account e otimização financeira.

A certificação AWS Certified DevOps Engineer - Professional valida habilidade avançada em provisionar, operar e gerenciar sistemas distribuídos na AWS, com foco em sistemas seguros, altamente disponíveis e escaláveis.

5.1 DevOps Professional — DOP-C02

Conteúdos

- CodePipeline avançado;
- multi-stage pipelines;
- cross-account deployment;

- manual approvals;
- artifact stores;
- quality gates;
- security gates;
- blue/green deployment;
- canary deployment;
- rolling deployment;
- CodeDeploy hooks;
- CloudFormation avançado;
- StackSets;
- drift detection;
- CDK Pipelines;
- constructs;
- infrastructure tests;
- Config aggregators;
- automated remediation;
- Incident Manager;
- runbooks;
- escalation plans;
- event-driven automation;
- OpsCenter;
- Change Manager.

Explicação técnica

CodePipeline, CodeBuild, CodeDeploy formam a base de entrega contínua na AWS. Servem para compilar, testar, validar, aprovar, publicar artefatos e fazer deploy.

Em projetos reais, são usados para APIs Java Spring Boot, Lambdas, containers em ECS/EKS e stacks CDK/CloudFormation.

Problemas que resolvem:

- deploy manual;
- falta de rastreabilidade;
- ausência de rollback;
- inconsistência entre ambientes;
- baixa qualidade de entrega.

Erros comuns:

- pipeline sem testes;
- deploy direto em produção;
- falta de aprovação manual em mudanças críticas;
- segredo dentro do pipeline;
- ausência de rollback;
- não separar contas por ambiente.

Visão sênior:

- desenhar pipeline multi-account;
- aplicar quality gate;
- aplicar security gate;
- exigir aprovação contextual;
- versionar artefatos;
- validar infraestrutura;
- ter rollback automático.
- Blue/green, canary e rolling deployment são estratégias de entrega.
- Rolling: substitui instâncias gradualmente.
- Blue/green: mantém dois ambientes e troca tráfego.
- Canary: libera para pequena parcela antes de expandir.

Trade-off:

- Rolling é simples e barato, mas rollback pode ser mais difícil.
- Blue/green facilita rollback, mas custa mais.
- Canary reduz risco, mas exige observabilidade madura.

Visão sênior: escolher estratégia por criticidade, custo, maturidade de monitoramento, impacto no usuário e capacidade de rollback.

CloudFormation, StackSets e drift detection controlam infraestrutura como código. StackSets permitem operações multi-account/multi-region com uma única operação, úteis para componentes compartilhados e controles de compliance.

Erro comum:

- alterar recurso manualmente no console;
- não detectar drift;
- não testar template;
- misturar aplicação e infraestrutura sem estratégia.

Visão sênior:

- evitar drift;
- separar stacks por domínio;
- criar módulos reutilizáveis;
- validar mudança antes de produção;
- aplicar guardrails.

Projeto prático

Pipeline cross-account:

- contas: dev, staging, prod;
- infraestrutura com CDK;
- deploy Java Spring Boot em ECS ou EKS;
- testes unitários;
- testes de integração;
- Sonar;

- Trivy;
- aprovação manual;
- deploy canary ou blue/green;
- rollback automático;
- notificações;
- logs de auditoria.

Critério sênior

Saber automatizar entrega com segurança, rastreabilidade, repetibilidade e rollback.

5.2 EKS e cloud-native AWS

Conteúdos

- EKS;
- Managed Node Groups;
- Fargate Profiles;
- Karpenter;
- IRSA;
- Cluster Autoscaler;
- VPC CNI;
- Security Groups for Pods;
- Network Policies;
- Ingress Controller;
- ALB Controller;
- ExternalDNS;
- Cert Manager;
- Secrets;
- Helm;
- ArgoCD;
- observabilidade no EKS;
- logs;
- métricas;
- traces;
- pod security;
- image scanning.

Explicação técnica

EKS é Kubernetes gerenciado na AWS. Serve para executar workloads containerizados com orquestração, escalabilidade e portabilidade.

Em projetos reais, EKS é usado quando há múltiplos microserviços, necessidade de GitOps, padronização de deploy, autoscaling sofisticado ou ecossistema Kubernetes maduro.

Erro comum: usar EKS para aplicações simples que ECS ou Lambda resolveriam com menor complexidade.

Visão sênior: decidir EKS apenas quando o ganho compensa a complexidade operacional.

Managed Node Groups simplificam gestão de nós EC2. Fargate Profiles executam pods sem gerenciar nós. Karpenter provisiona e remove nós conforme necessidade de scheduling, melhorando eficiência e custo.

Trade-off:

- Managed Node Groups: mais previsível.
- Fargate: menos operação, custo por pod.
- Karpenter: autoscaling avançado, exige maturidade.
- Self-managed nodes: mais controle, mais responsabilidade.
- IRSA permite que pods assumam roles IAM sem credenciais estáticas.

Erro comum: dar permissões amplas ao node role.

Visão sênior: permissões por workload, menor privilégio e auditoria por service account.

Ingress, ALB Controller, ExternalDNS e Cert Manager automatizam exposição de serviços, DNS e certificados.

Erro comum: expor serviço sem TLS, sem WAF, sem autenticação ou com rota pública desnecessária.

Helm e ArgoCD viabilizam empacotamento e GitOps.

Visão sênior:

- Git como fonte da verdade;
- promoção entre ambientes;
- rollback declarativo;
- revisão por pull request;
- separação de chart, values e secrets.
- Observabilidade no EKS exige logs, métricas e traces.

Erro comum:

- monitorar apenas o cluster, não os serviços;
- ignorar requests/limits;
- não observar HPA;
- não capturar erro por namespace/workload.

Visão sênior:

- definir SLO por serviço;
- coletar métricas de app e infra;
- aplicar tracing distribuído;
- monitorar custo por namespace;
- controlar limites, requests e autoscaling.

Projeto prático

Rodar a DevCore Platform em EKS com:

- Helm;
- ArgoCD;
- HPA;
- Ingress;
- TLS;
- Secrets;
- observabilidade;
- CI/CD;
- image scanning;
- namespaces por ambiente;
- políticas de segurança;
- rollback via GitOps.

Critério sênior

Saber operar Kubernetes sem transformar a plataforma em uma fonte permanente de complexidade, custo e incidentes.

5.3 Segurança AWS avançada

Conteúdos

- IAM avançado;
- Permission Boundaries;
- SCPs;
- ABAC;
- Cross-account roles;
- AWS Organizations;
- Control Tower;
- Security Hub;
- GuardDuty;
- Inspector;
- Macie;
- Detective;
- CloudTrail Lake;
- KMS;
- Secrets Manager;
- WAF;
- Shield;
- AWS Config;
- Security incident response.

Explicação técnica

IAM avançado envolve políticas condicionais, roles, boundaries, session policies, tags, federação e acesso temporário.

Visão sênior:

- reduzir permissões permanentes;
- usar roles cross-account;
- aplicar least privilege;
- revisar permissões com Access Analyzer;
- impedir escalonamento indevido.

SCPs são políticas do AWS Organizations que definem limites máximos de permissão para contas-membro. Elas não concedem acesso; funcionam como guardrails que restringem o que usuários e roles podem fazer dentro da organização.

Erro comum:

- aplicar SCP direto na raiz sem teste;
- bloquear serviços críticos;
- confundir SCP com política IAM;
- remover FullAWSAccess sem política substituta adequada.

Visão sênior:

- criar OUs por ambiente;
- aplicar SCP progressivamente;
- testar impacto;
- manter break-glass controlado;
- impedir regiões não aprovadas;
- impedir desativação de logs;
- impedir criação de recursos públicos sem aprovação.
- ABAC usa atributos/tags para controle de acesso.
- Exemplo: permitir que times gerenciem apenas recursos com Team=Payments.

Visão sênior: ABAC reduz explosão de policies, mas exige governança forte de tags.

Security Hub, GuardDuty, Inspector, Macie e Detective formam camada de detecção e postura de segurança.

GuardDuty: detecção de ameaças.

Inspector: vulnerabilidades.

Macie: dados sensíveis.

Security Hub: centralização de achados.

Detective: investigação.

Erro comum: ativar ferramentas sem processo de triagem e resposta.

Visão sênior:

- priorizar achados;
- integrar com ticket/alerta;

- criar remediação automática;
- medir tempo de correção;
- reduzir falso positivo.
- KMS e Secrets Manager protegem dados e credenciais.

Visão sênior:

- chaves por domínio;
- rotação;
- política de chave restritiva;
- auditoria de decrypt;
- segregação por ambiente;
- segredo nunca versionado no repositório.

Projeto prático

Baseline de segurança multi-account:

- Organizations;
- OUs;
- Control Tower ou landing zone equivalente;
- conta de audit;
- conta de log archive;
- CloudTrail organizacional;
- Config aggregator;
- Security Hub centralizado;
- GuardDuty;
- Inspector;
- Macie;
- SCPs;
- KMS;
- IAM Identity Center;
- roles cross-account;
- runbook de incidente.

Critério sênior

Saber montar baseline de segurança multi-account com governança, rastreabilidade, prevenção, detecção e resposta.

5.4 FinOps AWS

Conteúdos

- Cost Explorer;
- Budgets;
- Cost Anomaly Detection;

- Compute Optimizer;
- Savings Plans;
- Reserved Instances;
- Spot Instances;
- rightsizing;
- storage classes;
- S3 lifecycle;
- egress cost;
- Graviton;
- tagging strategy;
- unit economics.

Explicação técnica

Cost Explorer analisa custo por serviço, tag, conta, região e período.

Erro comum: olhar custo apenas no fim do mês.

Visão sênior: custo deve ser observado continuamente, como métrica operacional.

Budgets e Cost Anomaly Detection alertam sobre limite ou comportamento anormal.

Erro comum: configurar budget sem responsável pela ação.

Visão sênior: budget precisa ter owner, severidade e plano de resposta.

Compute Optimizer, rightsizing, Savings Plans, Reserved Instances e Spot reduzem custo de compute.

Trade-offs:

- Savings Plans reduzem custo, mas exigem compromisso.
- Reserved Instances podem ser boas para uso previsível.
- Spot reduz custo, mas pode ser interrompido.
- Rightsizing reduz desperdício, mas exige análise de performance.

Visão sênior: escolher economia sem prejudicar disponibilidade.

Storage classes e S3 lifecycle reduzem custo de armazenamento.

Erro comum: deixar tudo em S3 Standard para sempre.

Visão sênior:

- classificar dados por acesso;
- aplicar lifecycle;
- revisar retenção;
- controlar versionamento;
- evitar custo oculto de recuperação.
- Egress cost é custo de saída de dados.

Erro comum: ignorar tráfego entre regiões, NAT Gateway, internet e integrações externas.

Visão sênior:

- reduzir transferência desnecessária;

- usar VPC Endpoints;
- aproximar dados de compute;
- revisar arquitetura orientada a tráfego.
- Graviton pode reduzir custo/performance para workloads compatíveis.

Visão sênior: testar compatibilidade, benchmark e impacto em bibliotecas nativas.

Tagging strategy e unit economics conectam custo a negócio.

Exemplo:

- custo por ambiente;
- custo por feature;
- custo por cliente;
- custo por pedido;
- custo por transação;
- custo por microserviço.

Projeto prático

Dashboard de custo por:

- ambiente;
- serviço;
- time;
- feature;
- conta AWS;
- aplicação;
- centro de custo;
- custo por request/transação.

Critério sênior

Saber transformar custo cloud em decisão técnica e de produto.

Módulo 6 — Domínio sênior e liderança técnica

Objetivo do módulo

Consolidar todo o conhecimento em decisões arquiteturais, revisão técnica, governança, documentação, liderança e evolução contínua.

Conteúdos

- arquitetura multi-account;
- arquitetura multi-AZ;
- arquitetura event-driven;
- serverless vs containers;
- ECS vs EKS vs Lambda;
- RDS vs Aurora vs DynamoDB;

- ALB vs NLB vs API Gateway;
- CloudFormation vs CDK vs Terraform;
- observabilidade madura;
- disaster recovery;
- segurança por padrão;
- FinOps;
- revisão Well-Architected;
- documentação ADR;
- liderança técnica;
- mentoria;
- incident review;
- estratégia de evolução.

O que o sênior precisa fazer

- Um profissional sênior em AWS não é aquele que sabe o nome de muitos serviços. É aquele que consegue responder:
- Por que essa arquitetura é adequada?
- Qual problema ela resolve?
- Qual custo ela gera?
- Qual risco ela cria?
- Como ela falha?
- Como ela recupera?
- Como ela escala?
- Como ela é monitorada?
- Como ela é protegida?
- Como ela evolui?
- Como outro time consegue mantê-la?

Exercícios técnicos

- Criar ADRs para decisões cloud.
- Fazer revisão Well-Architected de um sistema real.
- Simular incidente de indisponibilidade.
- Fazer game day de falha de banco.
- Comparar ECS, EKS e Lambda para o mesmo workload.
- Desenhar arquitetura multi-account.
- Revisar IAM de um ambiente.
- Otimizar custo de um workload.
- Criar plano de DR.
- Apresentar arquitetura para banca técnica.

Mini projeto final

- Arquitetura completa de plataforma cloud:

- frontend em CloudFront/S3 ou Vercel integrado à AWS;
- API Java Spring Boot em ECS ou EKS;
- banco Aurora/RDS;
- cache Redis;
- SQS/EventBridge;
- S3 para arquivos;
- Secrets Manager;
- KMS;
- CI/CD cross-account;
- WAF;
- observabilidade;
- backup;
- DR;
- FinOps dashboard;
- documentação ADR;
- diagramas;
- runbooks;
- plano de incident response.

Critério de conclusão

- Você finaliza a trilha em nível sênior quando consegue:
- desenhar arquitetura completa;
- implementar infraestrutura como código;
- automatizar deploy;
- operar incidente;
- justificar custo;
- aplicar segurança;
- revisar arquitetura de outro dev;
- documentar decisões;
- orientar evolução técnica.

4. Critérios de domínio técnico por grande área

Cloud fundamentals

Iniciante: entende regiões, AZs, IAM básico, cobrança e responsabilidade compartilhada.

Intermediário: aplica IAM básico, budgets, CloudTrail, CloudWatch e tags.

Avançado: estrutura contas, monitora workloads e usa Well-Architected em revisões.

Sênior: define governança cloud, política de contas, padrões mínimos, riscos, custos e critérios arquiteturais.

Arquitetura AWS

Iniciante: conhece EC2, S3, RDS e VPC.

Intermediário: desenha aplicação com ALB, Auto Scaling, RDS, S3 e subnets privadas.

Avançado: inclui mensageria, cache, CDN, WAF, backup e DR.

Sênior: decide arquitetura com base em SLA, RTO, RPO, custo, segurança, operação e evolução.

Serverless e eventos

Iniciante: entende Lambda, API Gateway e SQS.

Intermediário: cria API serverless com DynamoDB e fila.

Avançado: usa EventBridge, Step Functions, DLQ, tracing e IaC.

Sênior: decide orquestração vs coreografia, controla idempotência, contratos de evento, custo por transição e falhas distribuídas.

Operação

Iniciante: lê logs e métricas.

Intermediário: cria alarmes e dashboards.

Avançado: usa X-Ray, Logs Insights, Config, Systems Manager e runbooks.

Sênior: reduz MTTR, define SLO, conduz incident response, postmortem e automação de remediação.

DevOps

Iniciante: entende CI/CD.

Intermediário: cria pipeline com build, teste e deploy.

Avançado: aplica quality gates, security gates e IaC.

Sênior: cria pipeline cross-account, rollback automático, deployment progressivo, auditoria e governança.

EKS

Iniciante: entende pods, services e deployments.

Intermediário: publica aplicação com Helm e Ingress.

Avançado: usa HPA, IRSA, ALB Controller, ExternalDNS, observabilidade e image scanning.

Sênior: decide ECS vs EKS, governa clusters, otimiza custo, segurança, autoscaling e GitOps.

Segurança

Iniciante: entende IAM, MFA e secrets.

Intermediário: aplica least privilege, KMS e WAF.

Avançado: usa SCPs, GuardDuty, Security Hub, Inspector, Macie e Config.

Sênior: desenha baseline multi-account, define guardrails, response plan, auditoria, detecção e remediação.

FinOps

Iniciante: entende billing.

Intermediário: usa budgets, tags e Cost Explorer.

Avançado: aplica rightsizing, Savings Plans, lifecycle e Compute Optimizer.

Sênior: conecta custo a produto, feature, cliente, workload e decisão arquitetural.

5. Aplicações práticas em sistemas reais

APIs

Aplicação:

- API Gateway ou ALB;
- Lambda, ECS ou EKS;
- autenticação;
- WAF;
- logs;
- métricas;
- throttling;
- tracing;
- deploy canary.

Decisão sênior: API Gateway é excelente para serverless e controle de API; ALB é comum para ECS/EKS/EC2; NLB é indicado quando há necessidade de camada 4, baixa latência ou TCP.

Banco de dados

Aplicação:

- RDS/Aurora para transacional;
- DynamoDB para alta escala e access patterns previsíveis;
- ElastiCache para cache;
- backups;
- criptografia;
- read replicas.

Decisão sênior: relacional é melhor para consistência transacional e consultas flexíveis; DynamoDB é excelente para escala e latência previsível, mas exige modelagem antecipada.

Microserviços

Aplicação:

- ECS/EKS;
- service discovery;
- SQS/EventBridge;
- logs centralizados;
- tracing;
- pipelines independentes;
- banco por serviço quando fizer sentido.

Decisão sênior: microsserviço sem observabilidade, ownership e deploy independente vira monólito distribuído problemático.

Mensageria

Aplicação:

- SQS para filas;
- SNS para fan-out;
- EventBridge para eventos de domínio;
- DLQ;
- retries;
- idempotência.

Decisão sênior: mensageria resolve acoplamento, mas cria complexidade de consistência eventual.

Segurança

Aplicação:

- IAM least privilege;
- KMS;
- Secrets Manager;
- WAF;
- GuardDuty;
- Security Hub;
- Config;
- SCPs.

Decisão sênior: segurança precisa ser preventiva, detectiva e responsiva.

Observabilidade

Aplicação:

- CloudWatch;
- Logs Insights;
- X-Ray;
- métricas customizadas;
- traces;
- dashboards;
- alarmes.

Decisão sênior: observabilidade não é “ter logs”, é conseguir responder rapidamente o que aconteceu, onde aconteceu, por que aconteceu e qual impacto gerou.

Testes

Aplicação:

- testes unitários;
- testes de integração;
- testes de contrato;
- testes de infraestrutura;

- testes de carga;
- smoke tests pós-deploy.

Decisão sênior: pipeline sem teste automatizado é automação de risco.

Deploy

Aplicação:

- rolling;
- blue/green;
- canary;
- feature flags;
- rollback;
- aprovação manual;
- quality/security gates.

Decisão sênior: estratégia de deploy depende de risco, criticidade, maturidade de monitoramento e custo.

Performance

Aplicação:

- cache;
- CDN;
- Auto Scaling;
- DynamoDB capacity;
- RDS tuning;
- ALB metrics;
- Lambda memory tuning;
- Graviton;
- redução de egress.

Decisão sênior: performance deve ser medida por latência, throughput, saturação, erro e custo por unidade.

Manutenção evolutiva

Aplicação:

- IaC;
- ADRs;
- versionamento;
- módulos reutilizáveis;
- ambientes separados;
- pipelines padronizados;
- revisão de arquitetura.

Decisão sênior: arquitetura boa é aquela que continua compreensível, segura e evolutiva depois de meses de mudanças.

6. Plano de execução sugerido

Duração recomendada

Para domínio real até nível avançado/sênior: 32 a 40 semanas.

Se o foco for certificação rápida, é possível reduzir. Mas para domínio de mercado, com projetos reais, a trilha precisa de prática consistente.

Semanas 1-3 — Fundamentos AWS

Prioridade: alta.

Estudar:

- cloud concepts;
- Regions;
- AZs;
- IAM;
- billing;
- CloudTrail;
- CloudWatch;
- Well-Architected.

Prática:

- conta AWS segura;
- MFA;
- budget;
- IAM roles;
- CloudTrail;
- alarmes.

Checkpoint:

- simulado CLF-C02;
- revisão dos seis pilares.

Semanas 4-10 — Arquitetura Associate

Prioridade: máxima.

Estudar:

- VPC;
- EC2;
- ALB/NLB;
- Auto Scaling;
- S3;
- RDS/Aurora;
- DynamoDB;

- CloudFront;
- Route 53;
- SQS/SNS/EventBridge;
- KMS;
- Secrets;
- Backup/DR.

Prática:

- arquitetura web/API com VPC;
- banco privado;
- ALB;
- Auto Scaling;
- S3;
- CloudFront;
- WAF;
- Backup.

Checkpoint:

- simulado SAA-C03;
- desenho arquitetural com justificativa.

Semanas 11-16 — Desenvolvimento AWS

Prioridade: alta.

Estudar:

- Lambda;
- API Gateway;
- DynamoDB;
- SQS;
- EventBridge;
- Step Functions;
- SAM/CDK;
- CloudWatch Logs;
- X-Ray;
- IAM least privilege.

Prática:

- API serverless completa;
- fila com DLQ;
- workflow com Step Functions;
- IaC com CDK;
- tracing.

Checkpoint:

- simulado DVA-C02;

- revisão de arquitetura serverless.

Semanas 17-21 — CloudOps / Operação

Prioridade: alta.

Estudar:

- CloudWatch avançado;
- Logs Insights;
- X-Ray;
- Systems Manager;
- Config;
- CloudTrail;
- VPC Flow Logs;
- troubleshooting;
- incident response.

Prática:

- dashboard operacional;
- runbooks;
- simulação de incidente;
- Config Rules;
- patching;
- troubleshooting RDS/ALB/VPC.

Checkpoint:

- simulado SOA-C03 / CloudOps Engineer Associate;
- postmortem técnico.

Semanas 22-27 — DevOps Professional

Prioridade: máxima para nível sênior.

Estudar:

- CodePipeline;
- CodeBuild;
- CodeDeploy;
- CDK Pipelines;
- CloudFormation;
- StackSets;
- cross-account;
- blue/green;
- canary;
- security gates;
- quality gates;
- rollback.

Prática:

- pipeline cross-account;
- deploy Spring Boot em ECS/EKS;
- Sonar;
- Trivy;
- aprovação manual;
- rollback automático.

Checkpoint:

- revisão estilo DOP-C02;
- apresentação da pipeline.

Semanas 28-32 — EKS e cloud-native

Prioridade: alta, especialmente para arquitetura moderna.

Estudar:

- EKS;
- nodes;
- Fargate;
- Karpenter;
- IRSA;
- Helm;
- ArgoCD;
- ALB Controller;
- observabilidade;
- pod security.

Prática:

- DevCore Platform em EKS;
- GitOps;
- HPA;
- Ingress;
- TLS;
- image scanning;
- secrets;
- dashboards.

Checkpoint:

- simulação de falha em pods/nodes;
- revisão de segurança do cluster.

Semanas 33-36 — Segurança avançada

Prioridade: máxima para sênior.

Estudar:

- IAM avançado;

- SCPs;
- Organizations;
- Control Tower;
- Security Hub;
- GuardDuty;
- Inspector;
- Macie;
- Detective;
- CloudTrail Lake;
- KMS;
- WAF;
- Config;
- incident response.

Prática:

- baseline multi-account;
- SCPs;
- centralização de logs;
- Security Hub;
- GuardDuty;
- runbook de segurança.

Checkpoint:

- revisão de segurança;
- simulação de credencial comprometida;
- plano de resposta.

Semanas 37-40 — FinOps e projeto final

Prioridade: alta.

Estudar:

- Cost Explorer;
- Budgets;
- Anomaly Detection;
- Compute Optimizer;
- Savings Plans;
- Reserved Instances;
- Spot;
- S3 lifecycle;
- egress;
- Graviton;
- tags;
- unit economics.

Prática:

- dashboard de custo;
- rightsizing;
- análise de egress;
- otimização de storage;
- custo por ambiente/feature.

Checkpoint final:

- projeto completo;
- documentação;
- ADRs;
- apresentação técnica;
- revisão Well-Architected.

7. Projetos recomendados para portfólio

Projeto 1 — API Java Spring Boot altamente disponível

Stack:

- Spring Boot;
- ECS ou EC2 Auto Scaling;
- ALB;
- RDS/Aurora;
- S3;
- Secrets Manager;
- CloudWatch;
- WAF;
- CI/CD.

Objetivo: demonstrar arquitetura tradicional bem feita.

Projeto 2 — API serverless orientada a eventos

Stack:

- API Gateway;
- Lambda;
- DynamoDB;
- SQS;
- EventBridge;
- Step Functions;
- CDK;
- X-Ray.

Objetivo: demonstrar domínio serverless e event-driven.

Projeto 3 — Pipeline DevOps cross-account

Stack:

- CodePipeline;
- CodeBuild;
- CodeDeploy;
- CDK;
- Sonar;
- Trivy;
- aprovação manual;
- rollback.

Objetivo: demonstrar entrega segura e profissional.

Projeto 4 — Plataforma em EKS

Stack:

- EKS;
- Helm;
- ArgoCD;
- HPA;
- ALB Controller;
- ExternalDNS;
- Cert Manager;
- IRSA;
- observabilidade.

Objetivo: demonstrar cloud-native e Kubernetes.

Projeto 5 — Baseline de segurança AWS

Stack:

- Organizations;
- SCP;
- Security Hub;
- GuardDuty;
- Inspector;
- Macie;
- Config;
- CloudTrail;
- KMS.

Objetivo: demonstrar governança e segurança multi-account.

Projeto 6 — FinOps dashboard

Stack:

- Cost Explorer;

- Budgets;
- tags;
- dashboards;
- análise por ambiente;
- análise por feature;
- unit economics.

Objetivo: demonstrar maturidade de custo e operação.

8. Checklist de domínio técnico

Fundamentos

- ☐ Sei explicar Region, AZ e Edge Location.
- ☐ Sei explicar responsabilidade compartilhada.
- ☐ Sei configurar IAM básico com segurança.
- ☐ Sei configurar budget e alerta de custo.
- ☐ Sei usar CloudTrail e CloudWatch.
- ☐ Sei aplicar os seis pilares do Well-Architected.

Arquitetura

- ☐ Sei desenhar VPC com subnets públicas e privadas.
- ☐ Sei configurar ALB/NLB.
- ☐ Sei escolher EC2, ECS, EKS ou Lambda.
- ☐ Sei escolher RDS, Aurora ou DynamoDB.
- ☐ Sei aplicar S3, CloudFront e Route 53.
- ☐ Sei desenhar backup e DR.
- ☐ Sei justificar custo, disponibilidade e segurança.

Desenvolvimento

- ☐ Sei criar API serverless.
- ☐ Sei modelar DynamoDB por access pattern.
- ☐ Sei usar SQS, SNS e EventBridge.
- ☐ Sei criar workflow com Step Functions.
- ☐ Sei usar SAM/CDK.
- ☐ Sei aplicar IAM least privilege.
- ☐ Sei investigar logs e traces.

Operação

- ☐ Sei criar dashboards e alarmes úteis.
- ☐ Sei usar Logs Insights.
- ☐ Sei usar Systems Manager.
- ☐ Sei aplicar Config Rules.
- ☐ Sei analisar VPC Flow Logs.

- ☐ Sei conduzir troubleshooting.
- ☐ Sei criar runbooks.

DevOps

- ☐ Sei criar pipeline multi-stage.
- ☐ Sei fazer deploy cross-account.
- ☐ Sei aplicar quality gates.
- ☐ Sei aplicar security gates.
- ☐ Sei usar blue/green, canary e rolling.
- ☐ Sei automatizar rollback.
- ☐ Sei versionar infraestrutura.

EKS

- ☐ Sei publicar aplicação em EKS.
- ☐ Sei usar Helm.
- ☐ Sei usar ArgoCD.
- ☐ Sei configurar Ingress e TLS.
- ☐ Sei aplicar IRSA.
- ☐ Sei configurar HPA.
- ☐ Sei monitorar pods, nodes e aplicações.

Segurança

- ☐ Sei aplicar SCPs.
- ☐ Sei desenhar multi-account.
- ☐ Sei usar GuardDuty, Security Hub e Inspector.
- ☐ Sei proteger secrets.
- ☐ Sei usar KMS corretamente.
- ☐ Sei criar baseline de segurança.
- ☐ Sei responder incidentes.

FinOps

- ☐ Sei analisar custo por serviço.
- ☐ Sei configurar Budgets.
- ☐ Sei aplicar tags.
- ☐ Sei fazer rightsizing.
- ☐ Sei analisar Savings Plans/Reserved/Spot.
- ☐ Sei reduzir custo de storage.
- ☐ Sei calcular custo por feature/transação.

9. Indicadores de prontidão para nível avançado/sênior

Você estará pronto para atuar em nível avançado/sênior quando conseguir:

- Desenhar arquitetura completa sem depender de tutorial.
- Justificar cada serviço escolhido.
- Explicar trade-offs de custo, segurança, performance e operação.
- Criar infraestrutura como código.
- Automatizar deploy com rollback.
- Operar incidentes com método.
- Projetar observabilidade antes do problema acontecer.
- Aplicar segurança por padrão.
- Estruturar contas AWS com governança.
- Otimizar custo com base em dados.
- Documentar decisões arquiteturais.
- Revisar arquitetura de outros desenvolvedores.
- Mentorar pessoas em decisões cloud.
- Defender tecnicamente uma arquitetura para negócio, produto, segurança e engenharia.

10. Resumo final da trilha

Esta trilha AWS forma um profissional com progressão real:

- Fundamentos cloud para entender a base.
- Arquitetura Associate para desenhar sistemas.
- Desenvolvimento AWS para construir aplicações cloud-native.
- Operação CloudOps para manter produção saudável.
- DevOps Professional para automatizar entrega segura.
- EKS/cloud-native para workloads modernos.
- Segurança avançada para governança multi-account.
- FinOps para controlar custo e eficiência.
- Liderança técnica para atuar como referência sênior.

O objetivo final não é apenas obter certificações, mas desenvolver domínio para projetar, operar, revisar e evoluir workloads críticos na AWS com maturidade de produção.

Uso recomendado: utilize este PDF como roteiro de estudo, checklist de domínio e base para montar projetos de portfólio com foco em produção.